VIVA GUIDE 04 / 05

Official documents and retrieval

The NIT Hamirpur PDF ingestion pipeline, lexical matching, chunking, and citations.

YOUR RESPONSIBILITY

Own the evidence pipeline: turn official institute PDFs into page-aware passages the answer service can retrieve.

THE SHORT VERSION

The official-document workflow downloads allowlisted PDFs from nith.ac.in, extracts text page by page, chunks it with overlap, and writes a local JSON index. At question time, a lightweight lexical retriever ranks passages and the service returns their document title, URL, and page.

A 30-SECOND OPENING

My part is the evidence pipeline. It makes the source of each answer traceable: official PDFs are converted into page-labelled text passages, matching passages are selected, and the title/page/URL travel with the answer.

IN THIS GUIDE

- Source set and data flow
- Safe, page-aware ingestion
- How lexical retrieval ranks evidence
- How source evidence reaches the student
- What the pipeline does not do
- Demo and viva questions

SOURCE SET AND DATA FLOW

The configured source list is ai-service/data/nith_sources.json. It currently names the NIT Hamirpur Hostel Booklet 2026-27, Odd Semester Academic Calendar 2026-27, Even Semester Academic Calendar 2026-27, and Fee Structure. The URLs are hosted on the official nith.ac.in domain.

```
nith_sources.json
|
v
scripts/ingest_nith_documents.py
HTTPS download -> PDF validation -> per-page text
-> overlapping chunks + document/page/URL metadata
-> ai-service/data/nith_documents.json
-> startup loader -> lexical retrieval -> answer sources
```

The generated nith_documents.json is local build output and is ignored by Git. The indexer can be rerun to recreate it. The AI service loads this file at startup; after rebuilding an index, restart the AI process so it reads the new file.

SAFE, PAGE-AWARE INGESTION

The ingestion script checks that each source uses HTTPS, the hostname is exactly nith.ac.in, and the path ends in .pdf. It sends a request with a user agent and a 60-second timeout, reads at most 30 MiB plus one byte to detect oversize downloads, and checks PDF content type or the %PDF- signature before parsing.

PyMuPDF opens the downloaded bytes in memory. Each page is extracted separately, whitespace is normalized, and chunks are created without merging page boundaries. Every chunk stores an id, document id, display title, source URL, 1-based page number, and excerpt text.

- Default chunk size is 500 words; default overlap is 100 words.
- The step is chunk size minus overlap (400 words), so neighboring chunks share context.
- Page metadata allows the UI and student to locate the source in the original PDF.
- The script errors when a configured PDF has no extractable text; it does not silently create an empty index.

The index is a local snapshot of selected official documents, not a live mirror of every NIT web page.

HOW LEXICAL RETRIEVAL RANKS EVIDENCE

retriever.py lowercases text, extracts a-z and 0-9 tokens, and removes a small stop-word list. For each passage it compares the unique query-token set with tokens in the passage's question/answer or text. The score is the count of shared query tokens divided by the query-token count.

The default similarity threshold is 0.65, but the implementation admits a passage when its overlap score is at least half the threshold (0.325 by default). Results sort first by overlap score and then by overlap with the document title; the top_k default is five.

- This is lexical matching: shared words matter. It does not use embeddings, a vector database, or semantic similarity.
- A paraphrase using different vocabulary can miss a relevant passage.
- Common words are removed, but the stop-word list is intentionally small.
- The threshold has a permissive half-threshold comparison; mention the actual rule rather than claiming it requires a 0.65 score.

HOW SOURCE EVIDENCE REACHES THE STUDENT

FastAPI formats each matched passage with its title, page, URL, and excerpt for the prompt. It also returns structured source entries with document id/title, URL, page, and relevance. Express stores those entries on the assistant message; React renders the title and page and links the title to the official PDF when a URL exists.

If the retriever finds no passage, FastAPI returns a fixed message that the requested information was not found in the indexed NIT documents. It returns no sources and skips Gemini. This is deliberately different from guessing based on model knowledge.

The prompt treats document text as reference material, not as instructions. That helps prevent text embedded in a document from overriding the assistant's grounding rules.

WHAT THE PIPELINE DOES NOT DO

- No OCR: image-only/scanned PDFs may yield no usable text.
- No automatic web refresh: source changes require rerunning ingestion and restarting the AI service.
- No vector embeddings or semantic index: retrieval is based on token overlap.
- No dynamic active-index update from admin upload: the upload/index endpoints do not currently parse files into this official JSON index.
- No claim that all institute policies are covered; the answer can only be as current and complete as the configured source set.

Before presenting an answer as authoritative, verify the cited PDF and page. The listed calendars and hostel booklet are year-specific; dates and rules can change. Direct students to the current official institute website for information outside the index.

DEMO AND VIVA QUESTIONS

- Open nith_sources.json and point out the official source allowlist.
- Run the ingestion script (network access required) and explain the generated local index.
- Open a passage record and identify document, page, URL, and excerpt.
- Ask a question using words in a source; compare it with a semantically similar paraphrase using different terms.
- Open a returned source link and verify the cited page in the PDF.

Why chunk documents instead of sending entire PDFs to Gemini?
Chunks keep prompts bounded and let the service select only a small amount of relevant evidence.

Why overlap chunks?
A fact near a chunk boundary can remain with neighboring context instead of being split away from its explanation.

Why preserve page numbers?
A page citation makes the answer verifiable in the source PDF.

Is this RAG?
It follows a retrieval-augmented generation pattern: retrieve source passages, then condition generation on them. Its retrieval is lexical, not vector-based.

What is the main retrieval weakness?
Word overlap can miss meaning-equivalent wording and can rank keyword matches that are not truly relevant.

How do you refresh source data?
Rerun the official ingestion script, verify the generated JSON, then restart FastAPI to reload it.

Does uploading an admin PDF add it to these passages?
No, that path currently stores metadata and sends a placeholder indexing request; it does not update the official index.

OFFICIAL DOCUMENTS REFERENCED

NIT Hamirpur Hostel Booklet 2026-27

NIT Hamirpur Academic Calendar, Odd Semester 2026-27

NIT Hamirpur Academic Calendar, Even Semester 2026-27

NIT Hamirpur Fee Structure